UNIT – V

Introduction to SQL:

SQL:
SQL (Structured Query Language) is the global standard programming language used to manage, query, and manipulate data stored in relational databases.

- Pure non-procedural query language
- Designed and developed by IBM, Implemented by Oracle
- 1978 System/R IBM - 1st Relational DBMS
- 1979 Oracle and Ingres
- 1982 SQL/DS and DB2 IBM
- Accepted by both ANSI + ISO as **Standard Query Language** for any RDBMS
- SQL86 (SQL1): first by ANSI and ratified by ISO (SQL-87), minor revision on 89 (SQL-89)
- SQL92 (SQL2): major revision
- SQL99 (SQL3): add recursive query, trigger, some OO features, and non-scholar type
- SQL2003: XML, Window functions, and sequences (Not free)
- Supports all the three sublanguages of DBMS: **DDL, DML, DCL**
- Supports Aggregate functions, String Manipulation functions, Set theory operations, Date Manipulation functions, rich set of operators (**IN, BETWEEN, LIKE, IS NULL, EXISTS**)
- Supports REPORT writing features and Forms for designing GUI based applications.

some key features of Structured Query Language (SQL):

1. Data Definition Language (DDL): SQL provides a set of commands to define and modify the structure of a database, including creating tables, modifying table structure, and dropping tables.
2. Data Manipulation Language (DML): SQL provides a set of commands to manipulate data within a database, including adding, modifying, and deleting data.
3. Query Language: SQL provides a rich set of commands for querying a database to retrieve data, including the ability to filter, sort, group, and join data from multiple tables.
4. Transaction Control: SQL supports transaction processing, which allows users to group a set of database operations into a single transaction that can be rolled back in case of failure.
5. Data Integrity: SQL includes features to enforce data integrity, such as the ability to specify constraints on the values that can be inserted or updated in a table, and to enforce referential integrity between tables.
6. User Access Control: SQL provides mechanisms to control user access to a database, including the ability to grant and revoke privileges to perform certain operations on the database.
7. Portability: SQL is a standardized language, meaning that SQL code written for one database management system can be used on another system with minimal modification.

Features of SQL in short

Data Definition Language (DDL) Features

1. CREATE: Create tables, indexes, views
2. ALTER: Modify table structure
3. DROP: Delete tables, indexes, views
4. TRUNCATE: Delete all table data

Data Manipulation Language (DML) Features

1. INSERT: Add new data
2. UPDATE: Modify existing data
3. DELETE: Delete data

Data Query Language (DQL) Features

1. SELECT: Retrieve data
2. FROM: Specify tables
3. WHERE: Filter data
4. GROUP BY: Group data
5. HAVING: Filter grouped data
6. ORDER BY: Sort data
7. LIMIT: Limit result set

Data Control Language (DCL) Features

1. GRANT: Assign permissions
2. REVOKE: Revoke permissions

Query Features

1. Sub queries
2. Aggregate functions (SUM, AVG, COUNT)
3. Grouping sets
4. Window functions
5. Common table expressions (CTE)
6. Joining tables (INNER, OUTER, FULL)
7. Union, Intersection, Difference

Data Integrity Features

1. Constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE)
2. Indexes (B-Tree, Hash)
3. Views
4. Stored Procedures
5. Functions

Transaction Control Features

1. COMMIT: Save changes
2. ROLLBACK: Undo changes
3. SAVEPOINT: Mark transaction point

Security Features

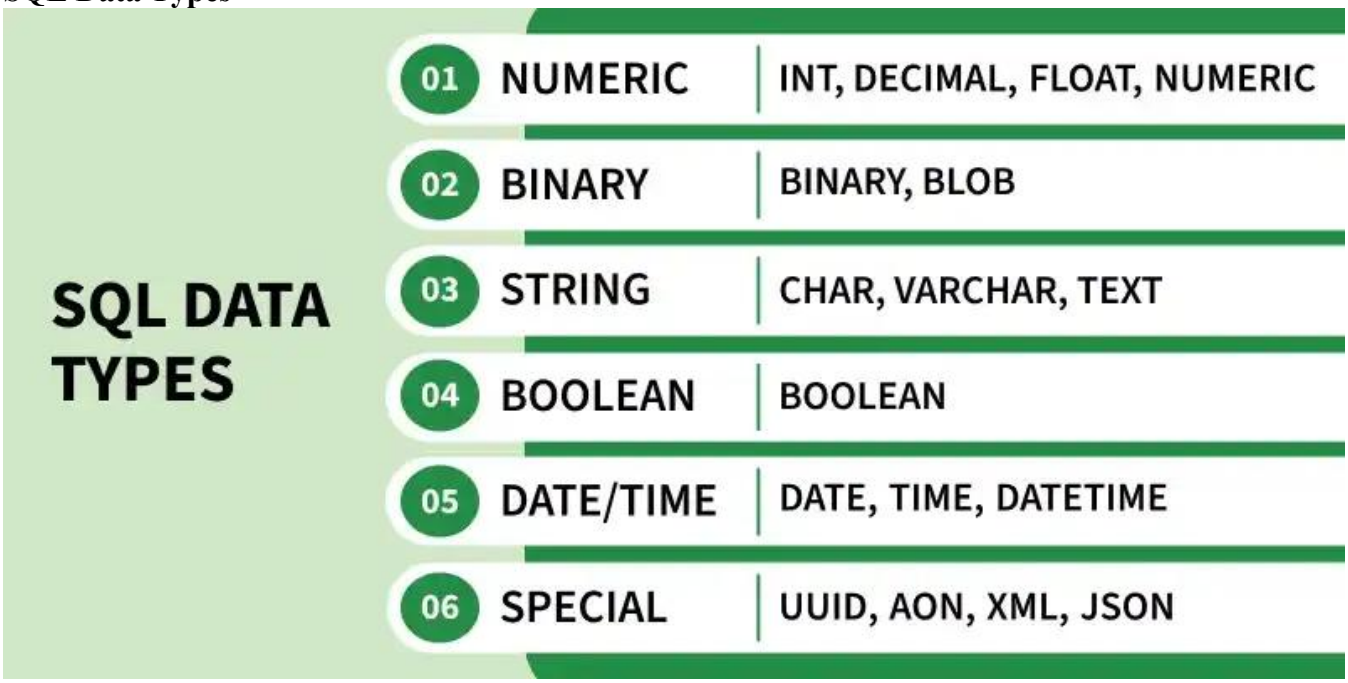
1. Authentication
2. Authorization
3. Encryption

Performance Optimization Features

1. Indexing
2. Caching
3. Query optimization

4. Partitioning

SQL Data Types



1. Numeric Data Types

Numeric data types are used to store integer and decimal values. They support mathematical operations and are commonly used for financial, scientific and analytical data.

Exact Numeric Datatype

Exact numeric types are used when precise numeric values are needed, such as for financial data, quantities and counts. Some common exact numeric types include:

Data Type	Description	Range
BIGINT	Large integer numbers	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
INT	Standard integer values	-2,147,483,648 to 2,147,483,647
SMALLINT	Small integers	-32,768 to 32,767
TINYINT	Very small integers	0 to 255
DECIMAL	Exact fixed-point numbers (e.g., for financial values)	-10^38 + 1 to 10^38 - 1

Data Type	Description	Range
NUMERIC	Similar to DECIMAL, used for precision data	-10^38 + 1 to 10^38 - 1

Example:

```
CREATE TABLE Product_Sales (
    ProductID INT PRIMARY KEY,
    Quantity SMALLINT,
    UnitPrice DECIMAL(10,2),
    TotalAmount DECIMAL(10,2)
);
```

Approximate Numeric Datatype

These types are used to store approximate values, such as scientific measurements or large ranges of data that don't need exact precision.

Data Type	Description	Range
FLOAT	Approximate numeric values	-1.79E+308 to 1.79E+308
REAL	Similar to FLOAT, but with less precision	-3.40E+38 to 3.40E+38

Example:

```
CREATE TABLE Measurements (
    SensorID INT,
    Temperature FLOAT,
    Humidity REAL
);
```

Data Type	Description
CHAR	Stores fixed-length non-Unicode characters with a maximum length of 8000 characters
VARCHAR	Stores variable-length non-Unicode characters with a maximum length of 8000 characters
VARCHAR(MAX)	Stores variable-length non-Unicode data with a maximum size of $2^{31} - 1$ characters (introduced in SQL Server 2005)
TEXT	Stores variable-length non-Unicode data with a maximum size of 2,147,483,647 characters

2. Character and String Data Types

Character data types are used to store text or character-based data. The choice between fixed-length and variable-length data types depends on the nature of your data.

Example:

```
CREATE TABLE Employee_Info (
   EmpID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName CHAR(30),
   Bio Nvarchar(max)
);
```

Unicode Character String Data Types

Unicode data types are used to store characters from any language, supporting a wider variety of characters. These are given in below table.

Data Type	Description
Nchar	The maximum length of 4000 characters(Fixed-Length Unicode Characters)
Nvarchar	The maximum length of 4000 characters.(Variable-Length Unicode Characters)
Nvarchar(max)	The maximum length of 2^31 - 1 characters(SQL Server 2005 only). (Variable Length Unicode data)

Example:

```
CREATE TABLE International_Users (
   UserID INT PRIMARY KEY,
   FullName NVARCHAR(100),
   Country NCHAR(50)
);
```

3. Date and Time Data Type

SQL provides several data types for storing date and time information. They are essential for managing timestamps, events and time-based queries. These are given in the below table.

Data Type	Description	Storage Size
DATE	Stores the data of date (year, month, day)	3 Bytes
TIME	Stores the data of time (hour, minute,second)	3 Bytes
DATETIME	Stores both the data and time (year, month, day, hour, minute, second)	8 Bytes

Example:

```
CREATE TABLE Orders (
   OrderID INT PRIMARY KEY,
```

```
    OrderDate DATE,
    OrderTime TIME,
    ShippedAt DATETIME
);
```

4. Binary Data Types in SQL

Binary data types are used to store binary data such as images, videos or other file types. These include

Data Type	Description	Max Length
Binary	Fixed-length binary data.	8000 bytes
VarBinary	Variable-length binary data.	8000 bytes
Image	Stores binary data as images.	2,147,483,647 bytes

Example:

```
CREATE TABLE Product_Images (
    ImageID INT PRIMARY KEY,
    ImageName VARCHAR(100),
    ImageData VARBINARY(MAX)
);
```

5. Boolean Data Type in SQL

Boolean data types store logical values such as TRUE and FALSE. In SQLite, these values are typically represented using integers. where:

- 1 represents TRUE
- 0 represents FALSE

Example:

```
CREATE TABLE User_Status (
    UserID INT PRIMARY KEY,
    IsActive INTEGER,
    IsVerified INTEGER
);
```

6. Special Data Types

SQL also supports some specialized data types for advanced use cases:

- **XML Data Type:** Used to store XML data and manipulate XML structures in the database

Example:

```
CREATE TABLE XML_Records (
    RecordID INT PRIMARY KEY,
    ConfigData XML
);
```

- **Spatial Data Type (Geometry):** stores planar spatial data, such as points, lines and polygons, in a database table.

Example:

```
CREATE TABLE Locations (
    LocationID INT PRIMARY KEY,
    Area GEOMETRY
);
```

SQL Constraints

SQL constraints are rules applied to columns or tables in a relational database to limit the type of data that can be inserted, updated, or deleted. These rules ensure the data is valid, consistent, and adheres to the business logic or database requirements. Constraints can be enforced during table creation or later using the ALTER TABLE statement. They play a vital role in maintaining the quality and integrity of your database.

Types of SQL Constraints

SQL provides several types of constraints to manage different aspects of data integrity. These constraints are essential for ensuring that data meets the requirements of accuracy, consistency, and validity. Let's go through each of them with detailed explanations and examples.

1. NOT NULL Constraint

The NOT NULL constraint ensures that a column cannot contain NULL values. This is particularly important for columns where a value is essential for identifying records or performing calculations. If a column is defined as NOT NULL, every row must include a value for that column.

Example:

```
CREATE TABLE Student
(
    ID int(6) NOT NULL,
    NAME varchar(10) NOT NULL,
    ADDRESS varchar(20)
);
```

Explanation: In the above example, both the ID and NAME columns are defined with the **NOT NULL** constraint, meaning every student must have an ID and NAME value.

2. UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column are distinct across all rows in a table. Unlike the PRIMARY KEY, which requires uniqueness and does not allow NULLs, the UNIQUE constraint allows NULL values but still enforces uniqueness for non-NULL entries.

Example:

```
CREATE TABLE Student
(
ID int(6) NOT NULL UNIQUE,
NAME varchar(10),
ADDRESS varchar(20)
);
```

Explanation: Here, the ID column must have unique values, ensuring that no two students can share the same ID. We can have more than one **UNIQUE** constraint in a table.

3. PRIMARY KEY Constraint

A PRIMARY KEY constraint is a combination of the NOT NULL and UNIQUE constraints. It uniquely identifies each row in a table. A table can only have one PRIMARY KEY, and it cannot accept NULL values. This is typically used for the column that will serve as the identifier of records.

Example:

```
CREATE TABLE Student
```

```
(
ID int(6) NOT NULL UNIQUE,
NAME varchar(10),
ADDRESS varchar(20),
PRIMARY KEY(ID)
);
```

Explanation: In this case, the ID column is set as the primary key, ensuring that each student's ID is unique and cannot be NULL.

4. FOREIGN KEY Constraint

A FOREIGN KEY constraint links a column in one table to the primary key in another table. This relationship helps maintain **referential integrity** by ensuring that the value in the foreign key column matches a valid record in the referenced table.

Orders Table:

O_ID	ORDER_NO	C_ID
1	2253	3
2	3325	3
3	4521	2
4	8532	1

Customers Table:

C_ID	NAME	ADDRESS
1	Robert Miller	New York
2	James Wilson	Los Angeles
3	Daniel Harris	Chicago

As we can see clearly that the field C_ID in Orders table is the primary key in Customers table, i.e. it uniquely identifies each row in the Customers table. Therefore, it is a Foreign Key in Orders table.

Example:

```
CREATE TABLE Orders
(
O_ID int NOT NULL,
ORDER_NO int NOT NULL,
C_ID int,
PRIMARY KEY (O_ID),
FOREIGN KEY (C_ID) REFERENCES Customers(C_ID)
)
```

Explanation: In this example, the C_ID column in the Orders table is a foreign key that references the C_ID column in the Customers table. This ensures that only valid customer IDs can be inserted into the Orders table.

5. CHECK Constraint

The CHECK constraint allows us to specify a condition that data must satisfy before it is inserted into the table. This can be used to enforce rules, such as ensuring that a column's value meets certain criteria (e.g., age must be greater than 18)

Example:

```
CREATE TABLE Student
(
ID int(6) NOT NULL,
NAME varchar(10) NOT NULL,
AGE int NOT NULL CHECK (AGE >= 18)
);
```

Explanation: In the above table, the CHECK constraint ensures that only students aged 18 or older can be inserted into the table.

6. DEFAULT Constraint

The DEFAULT constraint provides a default value for a column when no value is specified during insertion. This is useful for ensuring that certain columns always have a meaningful value, even if the user does not provide one

Example:

```
CREATE TABLE Student
(
ID int(6) NOT NULL,
NAME varchar(10) NOT NULL,
AGE int DEFAULT 18
);
```

Explanation: Here, if no value is provided for AGE during an insert, the default value of 18 will be assigned automatically.

How to Specify Constraints in SQL

Constraints can be specified during the table creation process using the CREATE TABLE statement. Additionally, constraints can be modified or added to existing tables using the ALTER TABLE statement.

Syntax for Creating Constraints:

CREATE TABLE table_name
(
column1 data_type [constraint_name],
column2 data_type [constraint_name],
column3 data_type [constraint_name],
...
);

We can also add or remove constraints after a table is created:

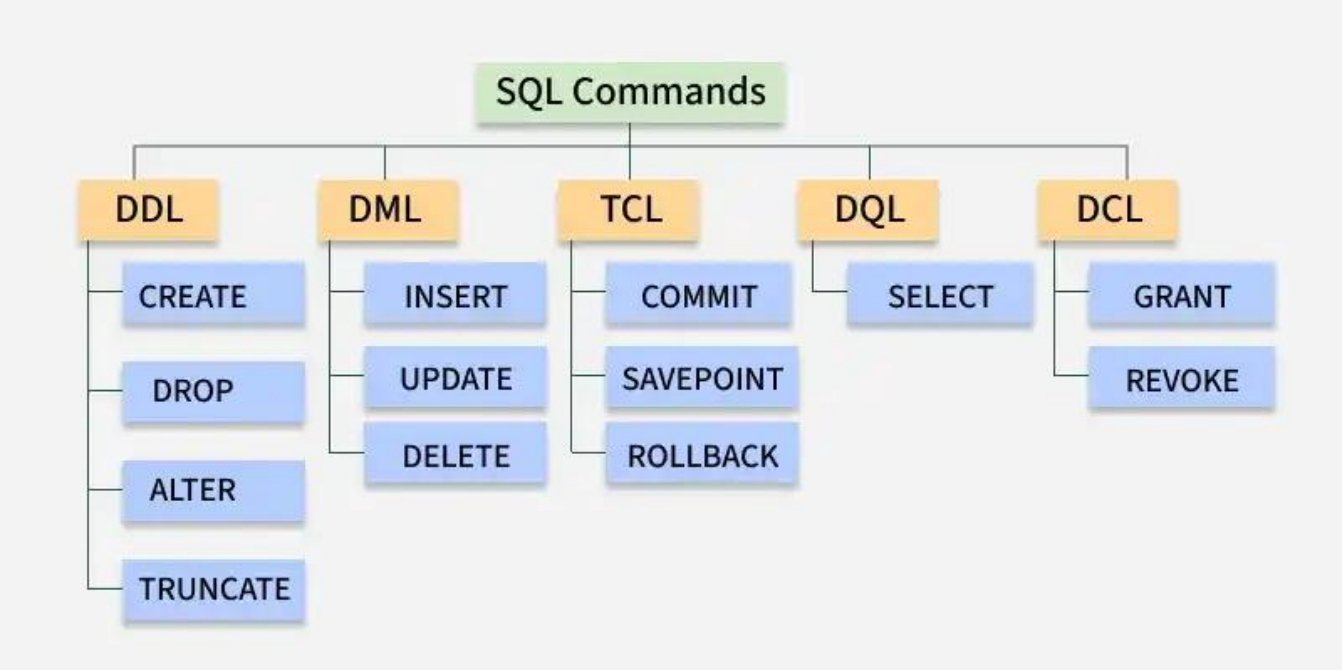
Example to Add a Constraint:

ALTER TABLE Student
ADD CONSTRAINT unique_student_id UNIQUE (ID);

SQL COMMANDS

SQL commands are fundamental building blocks used to perform given operations on database. The operations include queries of data. Creating a table, adding data to tables, dropping the table, modifying the table and setting permissions for users. Before learning SQL command types, it is important to understand some basic terms:

- **Database:** A collection of organized data.
- **Table:** Stores data in rows and columns.
- **Row (Record):** A single entry in a table.
- **Column (Field):** An attribute of the data.
- **Primary Key:** Uniquely identifies each record.
- **Query:** A request to access or modify data.
- **Constraint:** Rules applied to ensure valid data.



1. DDL - Data Definition Language

DDL (Data Definition Language) consists of SQL commands that can be used for defining, altering and deleting database structures such as tables, indexes and schemas. It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database

Command	Description	Syntax
CREATE	Create database or its objects (table, index, function, views, stored procedure and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Delete objects from the database	DROP TABLE table_name;
ALTER	Alter the structure of the database	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	TRUNCATE TABLE table_name;
COMMENT	Add comments to the data dictionary	COMMENT ON TABLE table_name IS 'comment_text';
RENAME	Rename an object existing in the database	RENAME TABLE old_table_name TO new_table_name;

Example:

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    hire_date DATE
);
```

In this example, a new table called employees is created with columns for employee ID, first name, last name and hire date.

2. DQL - Data Query Language

DQL is used to fetch data from the database. The main command is SELECT, which retrieves records based on the query. The output is returned as a result set (a temporary table) that can be viewed or used in applications.

Command	Description	Syntax
SELECT	It is used to retrieve data from the database	SELECT column1, column2, ...FROM table_name WHERE condition;
FROM	Indicates the table(s) from which to retrieve data.	SELECT column1 FROM table_name;
WHERE	Filters rows before any grouping or aggregation	SELECT column1 FROM table_name WHERE condition;
GROUP BY	Groups rows that have the same values in specified columns.	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1;
HAVING	Filters the results of GROUP BY	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1 HAVING condition;
DISTINCT	Removes duplicate rows from the result set	SELECT DISTINCT column1, column2, ... FROM table_name;
ORDER BY	Sorts the result set by one or more columns	SELECT column1 FROM table_name ORDER BY column1 [ASC | DESC];
LIMIT	Used to restrict the number of rows returned in a SELECT query (commonly supported in MySQL and PostgreSQL).	SELECT * FROM table_name LIMIT number;

Note: *DQL has only one command, SELECT. Other terms like FROM, WHERE, GROUP BY, HAVING, ORDER BY, DISTINCT and LIMIT are clauses of SELECT, not separate commands.*

Example:
SELECT first_name, last_name, hire_date
FROM employees
WHERE department = 'Sales'
ORDER BY hire_date DESC;

This query retrieves employees first and last names, along with their hire dates, from the employees table, specifically for those in the 'Sales' department, sorted by hire date.

3. DML - Data Manipulation Language
DML commands are used to manipulate the data stored in database tables. With DML, you can insert new records, update existing ones, delete unwanted data or retrieve information.

Command	Description	Syntax
INSERT	Insert data into a table	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
UPDATE	Update existing data within a table	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
DELETE	Delete records from a database table	DELETE FROM table_name WHERE condition;

Example:
INSERT INTO employees (first_name, last_name, department)
VALUES ('Jane', 'Smith', 'HR');

This query inserts a new record into employees table with first name 'Jane', last name 'Smith' and department 'HR'.

4. DCL - Data Control Language
DCL (Data Control Language) includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions and other controls of the database system. These commands are used to control access to data in the database by granting or revoking permissions.

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions or functions.	GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];
REVOKE	Removes previously granted privileges from a user account,	REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON

Command	Description	Syntax
	taking away their access to certain database objects or actions.	[object_type] object_name FROM user [CASCADE];

Example:
GRANT SELECT, UPDATE ON employees TO user_name;
This command grants the user user_name the permissions to select and update records in the employees table.

5. TCL - Transaction Control Language
Transactions group a set of tasks into a single execution unit. Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed. If any of the tasks fail, transaction fails. Therefore, a transaction has only two results: success or failure.

Command	Description	Syntax
BEGIN TRANSACTION	Starts a new transaction	BEGIN TRANSACTION [transaction_name];
COMMIT	Saves all changes made during the transaction	COMMIT;
ROLLBACK	Undoes all changes made during the transaction	ROLLBACK;
SAVEPOINT	Creates a savepoint within the current transaction	SAVEPOINT savepoint_name;

Example:
BEGIN TRANSACTION;
UPDATE employees SET department = 'Marketing' WHERE department = 'Sales';
SAVEPOINT before_update;
UPDATE employees SET department = 'IT' WHERE department = 'HR';
ROLLBACK TO SAVEPOINT before_update;
COMMIT;

In this example, a transaction is started, changes are made and a savepoint is set. If needed, the transaction can be rolled back to the savepoint before being committed.

DDL(Data Definition Language):
SQL CREATE TABLE

The CREATE TABLE statement in SQL is used to define a new table in a database. It specifies the table structure including column names and their data types.

- It is used to create a new table in a database.
- It defines column names and their data types.
- It helps in organizing and storing data systematically

Syntax:

```
CREATE TABLE table_name
(
Column1 datatype (size) constraint,
column2 datatype (size) constraint,
.
.
columnN datatype(size)
);
```

- **table_name**: The name you assign to the new table.
- **column1, column2, ...** : The names of the columns in the table.
- **datatype(size):** Defines the data type and size of each column.

Example: Create a Customer Table

Let's walk through a practical example where we create a Customer table that stores customer data. We will define various columns such as CustomerID, FirstName,lastName , Country, Age and Phone with appropriate data types and constraints.

Query:

```
CREATE TABLE Customer(
   CustomerID INT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50),
   Country VARCHAR(50),
   Age INT CHECK (Age >= 0 AND Age <= 99),
   Phone int(10)
);
```

Output:

CustomerID	FirstName	LastName	Country	Age	Phone
empty					

- CustomerID is an integer and serves as the PRIMARY KEY, ensuring each record is unique.
- FirstName, LastName and Country are VARCHAR fields to store variable-length text.
- Age has a CHECK constraint, ensuring it's within the range of 0 to 99.
- Phone is an integer field, although in real scenarios, a VARCHAR would often be used for

storing phone numbers to allow for leading zeros and formatting.

SQL DROP TABLE:

The DROP TABLE command permanently removes a table from the database.

- Deletes all rows and dependent objects, including indexes, constraints and triggers.
- Recovery is possible only through a backup or database recovery mechanism.

Syntax:
DROP TABLE table_name;

The following example demonstrates how to use the DROP TABLE command.

```
categories (
   CategoryID INT NOT NULL PRIMARY KEY,
   CategoryName NVARCHAR(50) NOT NULL,
   ItemDescription NVARCHAR(50) NOT NULL
);

INSERT INTO categories (CategoryID, CategoryName, ItemDescription)
VALUES
(1, 'Beverages', 'Soft Drinks'),
(2, 'Condiments', 'Sweet and Savory Sauces'),
(3, 'Confections', 'Sweet Breads');

SELECT * FROM categories;
```

Output:

CategoryID	CategoryName	ItemDescription
1	Beverages	Soft Drinks
2	Condiments	Sweet and Savory Sauces
3	Confections	Sweet Breads

Categories Table

At this point, the categories table has been created with three rows of sample data.

Step2: Drop the Table
Now, let's use the DROP TABLE statement to delete the categories table permanently

Query:

```
DROP TABLE categories;
```

Output:

SQL query successfully executed. However, the result set is empty.

Drop Categories Table

Working of SQL DROP TABLE

The DROP TABLE statement permanently removes a table from the database along with all its associated data and objects.

- Deletes the table, including its data, indexes, triggers, constraints and permissions.
- Once a table is dropped, it cannot be recovered, so this command should be used with caution.
- Using DROP TABLE IF EXISTS prevents errors by ensuring the table is dropped only if it exists.

DROP TABLE IF EXISTS categories;

- When a partitioned table is dropped, the table definition, all partitions, their data and partition metadata are removed. The partitioning scheme is also deleted if it is not used by other tables.
- Temporary tables can be dropped using the TEMPORARY keyword.

DROP TEMPORARY TABLE temp_table_name;

- To verify whether a table has been dropped, use commands like SHOW TABLES (MySQL) or query INFORMATION_SCHEMA.TABLES. If the table no longer exists, DESCRIBE or DESC will return an error.

SHOW TABLES;

SQL ALTER TABLE

The SQL ALTER TABLE statement is used to modify an existing table's structure without deleting it. It helps update the design of a database as requirements change.

- Can add, delete or modify columns in a table.
- Can also rename a table or change data types and constraints.
- Useful for adjusting database structure without losing data.

Example: First, we will create a demo SQL database and Employees table, on which we will use the ALTER TABLE command.

Employees

ID	Name	Department	Salary
1	John	HR	50000
2	Emma	IT	60000

Employees Table

Query:
ALTER TABLE Employees RENAME TO Staff;

Output:

Staff

ID	Name	Department	Salary
1	John	HR	50000
2	Emma	IT	60000

Output after renaming table
Syntax:
ALTER TABLE table_name [ADD | DROP | MODIFY] column_name datatype;

- **table_name:** name of the table you want to modify.
- **ADD:** used to add a new column.
- **DROP:** used to remove an existing column.
- **MODIFY:** used to change datatype or definition of an existing column.

Use Cases for SQL ALTER TABLE
Here are several usages of SQL ALTER TABLE
1. ADD
The ADD clause is used to add a new column to an existing table. You must specify the name of the new column and its data type.
Syntax:

```
ALTER TABLE table_name
ADD column_name datatype;
```

Query:

```
ALTER TABLE Staff
ADD Email VARCHAR(100);
```

Output:

Staff

ID	Name	Department	Salary	Email
1	John	HR	50000	
2	Emma	IT	60000	

Output after adding column
Here, we are adding a column named Email to Stafftable

2. MODIFY
The MODIFY (or ALTER COLUMN in some databases like SQL Server) clause is used to modify the definition of an existing column, such as changing its data type or size.

Syntax:

ALTER TABLE table_name
MODIFY COLUMN column_name datatype;

Query:

ALTER TABLE Staff
MODIFY COLUMN Address VARCHAR(100);

Here, we are modifying the column named Address datatype that is VARCHAR(100).

3. DROP

The DROP clause allows you to remove a column from a table. Be cautious when using this command as it will permanently remove the column and its data.

Syntax:

ALTER TABLE table_name
DROP COLUMN column_name;

Query:

ALTER TABLE Staff
DROP COLUMN Grade;

Here, we are removing a column named Grade from Staff table

4. RENAME COLUMN

We can rename an existing column using RENAME COLUMN clause. This allows you to change the name of a column while preserving its data type and content.

Syntax:

ALTER TABLE table_name
RENAME COLUMN old_name TO new_name;

Query:

ALTER TABLE Staff
RENAME COLUMN Name TO EmployeeName;

Output:

Staff

ID	EmployeeName	Department	Salary	Email
1	John	HR	50000	
2	Emma	IT	60000	

Output after renaming column

This renames the column Name to EmployeeName in the Staff table.

5. RENAME TO

We can rename an entire table using the RENAME TO clause. This changes the name of the table while preserving its structure and data.

Syntax:

ALTER TABLE table_name
RENAME TO new_table_name;

Query:

ALTER TABLE Staff

RENAME TO Employees;

Output:

Employees

ID	EmployeeName	Department	Salary
1	John	HR	50000
2	Emma	IT	60000

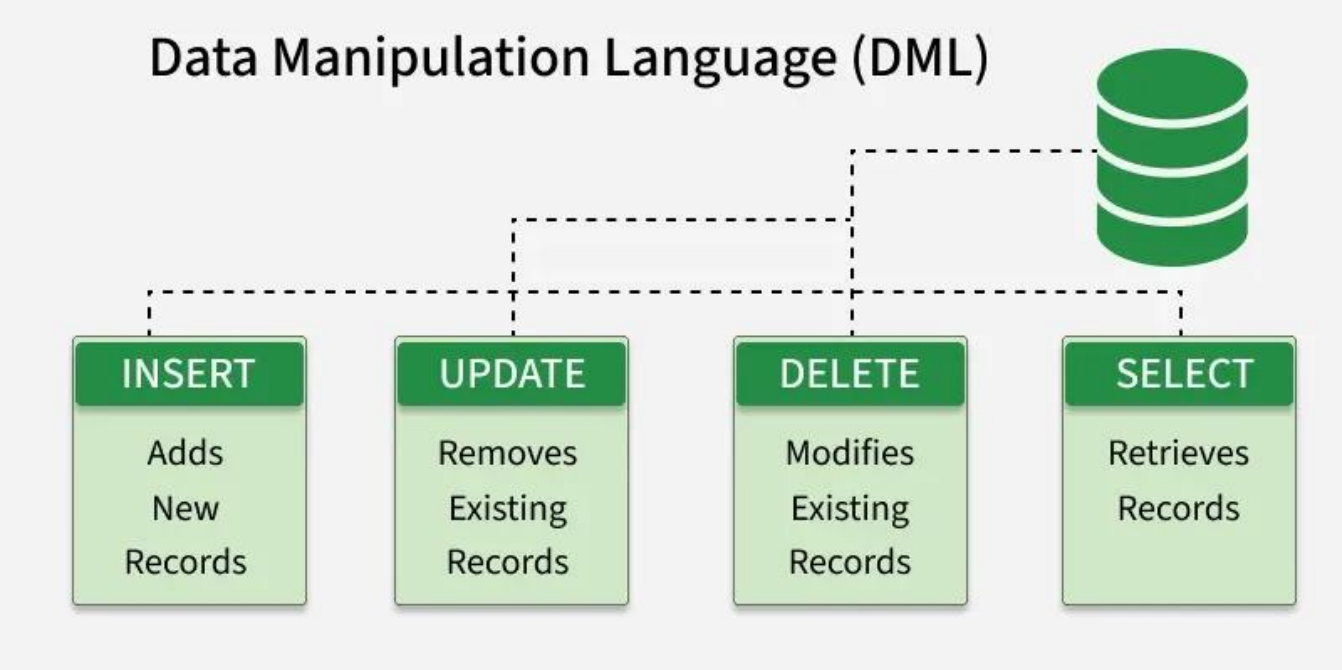
Output after renaming table name

This renames the table from Staff to Employees.

DML (Data Manipulation Language)

DML stands for Data Manipulation Language. Tables and formulas are helpful when communicating with data stored up to a point in a database through SQL (Structured Query Language), but a time comes when we actually want to execute some fairly complicated data interactions. DML is a way to inform a database precisely what we want it to do by conversing in a manner that it has been built to comprehend from scratch.

- DML provides an easy way to work with existing data, whether you want to add, update, move, or delete it.
- Every change made by a DML statement is treated as a transaction, so it requires transaction control.
- DML is a subset of SQL that modifies the actual data stored in tables.
- It focuses on database performance and works efficiently with systems like HDFS that follow an append-only storage model.



Generally, DML commands fall into one of four primary categories:

Generally, DML commands fall into one of four primary categories:

- **INSERT** adds fresh data to a table.
- **UPDATE** Change the data that is already in a table.
- **DELETE** takes a record out of a table.
- **SELECT** Get information out of one or more tables.

Types of Data Manipulation Language

There are basically two types of Data Manipulation Language. These are mentioned below. We have described them in the difference between format.

- High-Level or Non-Procedural DML
- Low-Level or Procedural DML

High-Level or Non-Procedural DML vs Low-level or Procedural DML

High-Level or Non-Procedural DML	Low-level or Procedural DML
It is also labelled as set-at-a-time or series oriented DML.	It is also labelled as track-at-a-time DML.
It can be used on its own for precisely specifying complex operations in the database.	It must be integrated to a general-purpose programming language.
It is prescriptive in nature.	It is indispensable in nature.
It demands that a user must clearly state which data is needed without clarifying how and when to obtain those data.	It demands that a user must clearly state which data is needed and how to obtain those data.
For Example: Every SQL statement is a prescriptive command.	**For Example:** DB2's SQL PL, Oracle's PL/SQL.

Characteristics of DML

It performs interpret-only data queries. It is used in a database schema to recall and manipulate the information. DML It is a dialect which is used to select, insert, delete and update data in a database. Data Manipulation Language (DML) commands are as follows:

SELECT Command

This command is used to get data out of the database. It helps users of the database to access from an operating system, the significant data they need. It sends a track result set from one tables or more.

Syntax :

```
SELECT *
FROM <table_name>;
```

Example:

```
SELECT *
FROM students;
```

OR

```
SELECT *
FROM students
where due_fees <=20000;
```

INSERT Command

This command is used to enter the information or values into a row. We can connect one or more records to a single table within a repository using this instruction. This is often used to connect an unused tag to the documents.

Syntax:

```
INSERT INTO <table_name> ('column_name1' <datatype>, 'column_name2' <datatype>)
VALUES ('value1', 'value2');
```

Example :

```
INSERT INTO students ('stu_id' int, 'stu_name' varchar(20), 'city' varchar(20))
VALUES ('1', 'Nirmit', 'Gorakhpur');
```

UPDATE Command

This command is used to alter existing table records. Within a table, it modifies data from one or more records. This command is used to alter the data which is already present in a table.

Syntax:

```
UPDATE <table_name>
SET <column_name = value>
WHERE condition;
```

Example:

```
UPDATE students
SET due_fees = 20000
WHERE stu_name = 'Mini';
```

DELETE Command

It deletes all archives from a table. This command is used to erase some or all of the previous table's records. If we do not specify the 'WHERE' condition then all the rows would be erased or deleted.

Syntax:

```
DELETE FROM <table_name>
WHERE <condition>;
```

Example:

```
DELETE FROM students
WHERE stu_id = '001';
```

Benefits of DML

- DML statements could alter the data that is contained or stored in the database.
- It delivers effective human contact with the machine.
- User could specify what data is required.
- DML aims to have many different varieties and functionalities between vendors providing databases.

Limitations of DML

- We cannot use DML to change the structure of the database.
- Limit table view i.e., it could conceal some columns in tables.

- Access the data without having the data stored in the object.
- Unable to build or erase lists or sections using DML.

Queries: Simple Queries, select with WHERE clause

The SQL SELECT statement is used to retrieve data from one or more tables and display it in a structured format of rows and columns.

- Fetches all columns using * or specific columns by name.
- Filters and sorts data using WHERE and ORDER BY.
- Supports grouping, aggregation and table relationships using GROUP BY, HAVING and JOIN.

Example: First, we create a demo SQL table, on which we use the SELECT query.

CustomerID	FirstName	LastName	Country	Age	Phone
1	Liam	Smith	USA	23	xxxxxxxxxx
2	Sophia	Miller	USA	21	xxxxxxxxxx
3	Akira	Tanaka	Japan	24	xxxxxxxxxx
4	Carlos	Hernandez	USA	21	xxxxxxxxxx
5	Isabella	Rossi	Italy	22	xxxxxxxxxx

Customer Table

Query:

```
SELECT CustomerID, FirstName FROM Customer;
```

Output:

CustomerID	FirstName
1	Liam
2	Sophia
3	Akira
4	Carlos
5	Isabella

Syntax:

```
SELECT column1,column2.... FROM table_name;
```

- **column1, column2:** columns you want to retrieve.
- **table_name:** name of the table you're querying.

Working with the SELECT Statement

The SELECT statement is used to retrieve and display data from one or more tables in a structured and readable format.

Example 1: Select Specific Columns

In this example, we will demonstrate how to retrieve specific columns from the Customer table. Here we will fetch only FirstName and LastName for each record.

Query:

```
SELECT FirstName, LastName
FROM Customer;
```

Output:

FirstName	LastName
Liam	Smith
Sophia	Miller
Akira	Tanaka
Carlos	Hernandez
Isabella	Rossi

Example 2: Select All Columns

In this example, we will fetch all the fields from table Customer:

Query:

```
SELECT * FROM Customer;
```

Output:

CustomerID	FirstName	LastName	Country	Age	Phone
1	Liam	Smith	USA	23	xxxxxxxxxx
2	Sophia	Miller	USA	21	xxxxxxxxxx
3	Akira	Tanaka	Japan	24	xxxxxxxxxx
4	Carlos	Hernandez	USA	21	xxxxxxxxxx
5	Isabella	Rossi	Italy	22	xxxxxxxxxx

Example 3: SELECT Statement with WHERE Clause

Suppose we want to see table values with specific conditions then WHERE Clause is used with select statement. In this example, filter customers who are 21 years old.

Query:

```
SELECT FirstName
FROM Customer
WHERE Age = 21;
```

Output:

FirstName
Sophia
Carlos

Example 4: SELECT with GROUP BY Clause

In this example, we will use SELECT statement with GROUP BY Clause to group rows and perform aggregation. Here, we will count the number of customers from each country.

Query:

```
SELECT Country, COUNT(*) AS customer_count
FROM Customer
GROUP BY Country;
```

Output:

Country	customer_count
Italy	1
Japan	1
USA	3

Example 5: SELECT with DISTINCT Clause

In this example, we will use DISTINCT keyword to return only unique values from a column. Here, we will fetch unique countries from the Customer table.

Query:

```
SELECT DISTINCT Country
FROM Customer;
```

Output:

Country
USA
Japan
Italy

Example 6: SELECT Statement with HAVING Clause

The HAVING clause is used to filter results after applying GROUP BY. In this example, we will find countries that have 2 or more customers in the Customer table.

Query:

```
SELECT Country, COUNT(*) AS customer_count
FROM Customer
GROUP BY Country
HAVING COUNT(*) >= 2;
```

Output:

Country	customer_count
USA	3

Example 7: SELECT Statement with ORDER BY clause
In this example, we will use SELECT Statement with ORDER BY clause. Here, Sort results by Age in descending order.
Query:

```
SELECT * FROM Customer ORDER BY Age DESC;
```

Output:

CustomerID	FirstName	LastName	Country	Age	Phone
3	Akira	Tanaka	Japan	24	xxxxxxxxxx
1	Liam	Smith	USA	23	xxxxxxxxxx
5	Isabella	Rossi	Italy	22	xxxxxxxxxx
2	Sophia	Miller	USA	21	xxxxxxxxxx
4	Carlos	Hernandez	USA	21	xxxxxxxxxx

SQL Aggregate functions

SQL Aggregate Functions allow summarizing large sets of data into meaningful results, making it easier to analyze patterns and trends across many records. They return a single output value after processing multiple rows in a table.

- Perform calculations like totals, averages, minimum or maximum values on data.
- Ignore NULL values in most functions except COUNT(*), improving result accuracy.
- Work with clauses such as GROUP BY, HAVING and ORDER BY for analysis.

Example: First, we create a demo SQL database and table, on which we use the Aggregate functions.

EmpID	Name	Salary
1	John	5000
2	Sara	
3	Michael	8000
4	Emma	5000

Employee Table

Query:

```
SELECT SUM(Salary) FROM Employee;
```

Output:

SUM(Salary)
18000

Syntax:

AGGREGATE_FUNCTION(column_name)

Aggregate Functions in SQL

Below are the most frequently used aggregate functions in SQL.

1. Count()

It is used to count the number of rows in a table. It helps summarize data by giving the total number of entries. It can be used in different ways depending on what you want to count:

- **COUNT(*):** Counts all rows.
- **COUNT(column_name):** Counts non-NULL values in the specified column.
- **COUNT(DISTINCT column_name):** Counts unique non-NULL values in the column.

Query:

```
-- Total number of records in the table
SELECT COUNT(*) AS TotalRecords FROM Employee;

-- Count of non-NULL salaries
SELECT COUNT(Salary) AS NonNullSalaries FROM Employee;

-- Count of unique non-NULL salaries
SELECT COUNT(DISTINCT Salary) AS UniqueSalaries FROM Employee;
```

Output:

TotalRecords
4

NonNullSalaries
3

UniqueSalaries
2

- COUNT(*) returns the total number of rows in the table, including rows with NULL values.
- COUNT(Salary) counts only the rows where Salary is not NULL.
- COUNT(DISTINCT Salary) counts unique non-NULL salary values, ignoring duplicates.

2. SUM()

It is used to calculate the total of a numeric column. It adds up all non-NULL values in that column. For Example, SUM(column_name) returns sum of all non-NULL values in the specified column.

Query:

```
-- Calculate the total salary
SELECT SUM(Salary) AS TotalSalary FROM Employee;

-- Calculate the sum of unique salaries
SELECT SUM(DISTINCT Salary) AS DistinctSalarySum FROM Employee;
```

Output:

TotalSalary
18000

DistinctSalarySum
13000

- SUM(Salary) adds all non-NULL salary values to get the total salary amount.
- SUM(DISTINCT Salary) adds only unique non-NULL salary values, avoiding duplicates.
- NULL values are ignored in both SUM calculations.

3. AVG()

It is used to calculate average value of a numeric column. It divides sum of all non-NULL values by the number of non-NULL rows. For Example, AVG(column_name) returns average of all non-NULL values in the specified column.

Query:

```
-- Calculate the average salary
SELECT AVG(Salary) AS AverageSalary FROM Employee;

-- Average of distinct salaries
SELECT AVG(DISTINCT Salary) AS DistinctAvgSalary FROM Employee;
```

Output:

AverageSalary
6000

DistinctAvgSalary
6500

- AVG(Salary) calculates the average of all non-NULL salary values.
- AVG(DISTINCT Salary) computes the average only from unique non-NULL salary values.
- Both ignore NULL values when performing the calculation.

4. MIN() and MAX()

The MIN() and MAX() functions return the smallest and largest values, respectively, from a column.

Query:

```
-- Find the highest salary
SELECT MAX(Salary) AS HighestSalary FROM Employee;

-- Find the lowest salary
SELECT MIN(Salary) AS LowestSalary FROM Employee;
```

Output:

HighestSalary
8000

LowestSalary
5000

- MAX(Salary) returns the highest non-NULL salary value from the Employee table.
- MIN(Salary) returns the lowest non-NULL salary value from the Employee table.
- Both functions ignore NULL values while determining the result.

The SQL GROUP BY Statement

Purpose: Groups rows with similar values into categories.
The GROUP BY statement is used to group rows that have the same values into summary rows, like "Find the number of customers in each country".
The GROUP BY statement is almost always used in conjunction with aggregate functions, like COUNT(), MAX(), MIN(), SUM(), AVG(), to perform calculations on each group.

GROUP BY Syntax:

SELECT ***column1, aggregate_function(column2), column3, ...***
FROM ***table_name***
WHERE ***condition***
GROUP BY ***column1, column3***
ORDER BY ***column_name*;**

Example:

Consider the following Employee table with data

employees

name	age	salary	dname
yyy	28	55000	coding
zzz	25	50000	coding
aaa	24	45000	testing
bbb	29	65000	testing

```
SELECT dname, SUM(salary)
FROM employees
GROUP BY dname;
```

dname	SUM(salary)
coding	105000
testing	110000

HAVING Clause:

Purpose: Filters grouped results based on conditions. It is used to apply condition to group.

Syntax:

```
SELECT column1, column2
FROM table_name
GROUP BY column1, column2
HAVING condition;
```

Example

```
SELECT dname, SUM(salary)
FROM employees
GROUP BY dname
Having SUM(salary)>106000;
```

dname	SUM(salary)
testing	110000

SQL ALL and ANY Operators

In SQL, the ALL and ANY operators are used to compare a value against a set of values returned by a subquery. They help filter results based on conditions evaluated over multiple values.

- ALL checks if a condition is true for every value in the subquery result.
- Must be used with comparison operators like >, <, >=, <=.
- Used in SELECT, WHERE and HAVING for advanced filtering.

SQL ALL

The SQL ALL operator compares a value to every value returned by a subquery. A condition using ALL is TRUE only if it holds for all values in the subquery result.

- Must be used with comparison operators: >, <, >=, <=.
- Useful for filtering data based on comparisons with multiple records.

Example: First, we will create a demo SQL database and table, on which we will use the ALL command.

ProductID	Name	Price
1	Laptop	800
2	Tablet	500
3	Phone	300

Products Table

Query:

```
SELECT * FROM Products
WHERE Price > ALL (SELECT Price FROM Products WHERE Price < 500);
```

Output:

ProductID	Name	Price
1	Laptop	800
2	Tablet	500

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name comparison_operator ALL
  (SELECT column_name
   FROM table_name
   WHERE condition(s));
```

- **comparison_operator:** This is the comparison operator that can be one of =, >, <, >=, <=, <>, etc.
- **subquery:** A query that returns the set of values to be compared with the column in the outer query.

SQL ALL with SELECT, WHERE & HAVING

The ALL operator can be used in conjunction with SELECT, WHERE and HAVING statements to refine your data filtering.

ProductID	ProductName	SupplierID	CategoryID	Price
1	Chais	1	1	18
2	Chang	1	1	19
3	Aniseed Syrup	1	2	10
4	Chef Anton's Cajun Seasoning	2	2	22
5	Chef Anton's Gumbo Mix	2	2	21
6	Boysenberry Spread	3	2	25
7	Organic Dried Pears	3	7	30
8	Northwoods Cranberry Sauce	3	2	40
9	Mishi Kobe Niku	4	6	97

Products Table

OrderDetailsID	OrderID	ProductID	Quantity
1	10248	1	12
2	10248	2	10
3	10248	3	15
4	10249	1	8
5	10249	4	4
6	10249	5	6
7	10250	3	5
8	10250	4	18
9	10251	5	2
10	10251	6	8
11	10252	7	9
12	10252	8	9
13	10250	9	20
14	10249	9	4

OrderDetails Table

Example 1 : Retrieve all product names from the Products table
Query:

```
SELECT ALL ProductName
FROM Products
WHERE TRUE;
```

Output:

ProductName
Chais
Chang
Aniseed Syrup
Chef Anton's Cajun Seasoning
Chef Anton's Gumbo Mix
Boysenberry Spread
Organic Dried Pears
Northwoods Cranberry Sauce
Mishi Kobe Niku

This query retrieves all product names from the Products table because TRUE always evaluates as true for every row.

Example 2: Retrieve product names if all records in the OrderDetails table have a quantity of 6 or 2

```
SELECT ProductName
FROM Products
WHERE ProductID = ALL (SELECT ProductID
                       FROM OrderDetails
                       WHERE Quantity = 6 OR Quantity = 2);
```

Output:

ProductName
Chef Anton's Gumbo Mix

This query ensures that the product names returned have ALL quantities of 6 or 2 in the OrderDetails table.

Example 3 : Find the OrderIDs where the maximum quantity in the order exceeds the average quantity of all orders

```
SELECT OrderID
FROM OrderDetails
GROUP BY OrderID
HAVING MAX(Quantity) > ALL (SELECT AVG(Quantity)
```

```
FROM OrderDetails
GROUP BY OrderID);
```

Output:

OrderID
10248
10250

This query filters out OrderIDs where the maximum quantity is greater than the average quantity of the orders.

SQL ANY Operator

The SQL ANY operator compares a value with the values returned by a subquery and evaluates to TRUE if at least one value satisfies the given condition.

- It must be used with comparison operators such as =, >, <, >=, <= or <>.
- The subquery used with ANY must return a single column of values.

Example: First, we will create a demo SQL database and table, on which we will use the ANY command.

ProductID	Name	Price
1	Laptop	800
2	Tablet	500
3	Phone	300

Query:

```
SELECT * FROM Products
WHERE Price < ANY (SELECT Price FROM Products WHERE Price > 500);
```

Output:

ProductID	Name	Price
2	Tablet	500
3	Phone	300

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name comparison_operator ANY
  (SELECT column_name
   FROM table_name
   WHERE condition(s));
```

SQL ANY with SELECT, WHERE & HAVING

The ANY operator can be used with SELECT, WHERE and HAVING clauses to filter data by matching a condition against any value in a subquery result.

Example 1 : Find distinct category IDs of products that appear in the OrderDetails table.

Query:

```
SELECT DISTINCT CategoryID
FROM Products
WHERE ProductID = ANY (SELECT ProductID
                       FROM OrderDetails);
```

Output:

CategoryID
1
2
7
6

This query finds the distinct CategoryIDs of products that exist in the OrderDetails table.

Example 2 : Find product names with a quantity of 9 in the OrderDetails table

Query:

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY (SELECT ProductID
                       FROM OrderDetails
                       WHERE Quantity = 9);
```

Output:

ProductName
Chais
Chang
Aniseed Syrup
Chef Anton's Cajun Seasoning
Chef Anton's Gumbo Mix
Boysenberry Spread
Organic Dried Pears
Northwoods Cranberry Sauce
Mishi Kobe Niku

This query retrieves product names where at least one record in the OrderDetails table has a quantity of 9.

LIKE Operator:

The SQL LIKE operator is used to search for a specific pattern within a column's text data. It works with wildcard characters to match partial strings, making it useful for flexible filtering.

- % represents any sequence of characters
- _ represents a single character

Example: First, we create a demo SQL database and table, on which we will use the LIKE Operator command.

EmpID	EmpName	Salary
1	Akira	48000
2	Luca	52000
3	Sofia	45000

Query:

```
SELECT * FROM Employees
WHERE EmpName LIKE 'A%';
```

Output:

EmpID	EmpName	Salary
1	Akira	48000

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE column_name LIKE pattern;
```

- column_name: The column to be searched.
- pattern: The pattern to search for, which can include wildcard characters.

Wildcard Characters with the SQL LIKE Operator

Wildcards are used with the LIKE operator to search for specific patterns in strings. Wildcard characters substitute one or more characters in the string. There are four wildcard characters in SQL:

1. **% (Percent):** Represents zero or more characters.
2. **_ (Underscore):** Represents a single character.
3. **[] (Square Brackets):** Represents any single character within brackets.
4. **- (Hyphen):** Specify a range of characters inside brackets.

Pattern	Meaning
'a%'	Match strings that start with 'a'
'%a'	Match strings that end with 'a'
'a%t'	Match strings that contain the start with 'a' and end with 't'.
'%wow%'	Match strings that contain the substring 'wow' in them at any position.
'_wow%'	Match strings that contain the substring 'wow' in them at the second position.
'_a%'	Match strings that contain 'a' at the second position.
'a_ _%'	Match strings that start with 'a and contain at least 2 more characters.

Examples of SQL LIKE Operator

Let's understand LIKE operator in SQL with examples. First, we will create a demo SQL database and table, on which we will use the SQL LIKE Operator command.

SupplierID	Name	Address
S1	Tokyo Trading Co.	3-15 Shibuya, Tokyo, Japan
S2	La Maison du Marché	12 Rue Lafayette, Paris, France
S3	Nordic Exports AB	45 Kungsgatan, Stockholm, Sweden
S4	Café del Sol Imports	Calle 22, Madrid, Spain
S5	Silver Maple Supplies	221 King Street, Toronto, Canada
S6	Bavaria Goods GmbH	89 Hauptstraße, Munich, Germany

Example 1 : Match Names Starting with 'Ca'

Retrieve SupplierID, Name and Address from suppliers table, where supplier name starts form Ca.

```
SELECT SupplierID, Name, Address
FROM Supplier
WHERE Name LIKE 'Ca%';
```

Output:

SupplierID	Name	Address
S4	Café del Sol Imports	Calle 22, Madrid, Spain

Example 2: Match Addresses Containing 'Kungsgatan'

Retrieve entire table, where address contains Kungsgatan.

```
SELECT *
FROM Supplier
WHERE Address LIKE '%Kungsgatan%';
```

Output:

SupplierID	Name	Address
S3	Nordic Exports AB	45 Kungsgatan, Stockholm, Sweden

Example 3: Match Names Where 'afé' Appears in the Second Position

Retrieve the SupplierID, Name and Address of suppliers whose Name has "afé" starting from the second position.

```
SELECT SupplierID, Name, Address
FROM Supplier
WHERE Name LIKE '_afé%';
```

Output:

SupplierID	Name	Address
S4	Café del Sol Imports	Calle 22, Madrid, Spain

Example 4: Using LIKE with AND for Complex Conditions

Retrieve suppliers from Madrid with names starting with "C":

```
SELECT SupplierID, Name, Address
FROM Supplier
WHERE Address LIKE '%Madrid%' AND Name LIKE 'C%';
```

Output:

SupplierID	Name	Address
S4	Café del Sol Imports	Calle 22, Madrid, Spain

Example 5: Using NOT LIKE for Exclusion

To retrieve all suppliers whose name does not contain "Co"

```
SELECT SupplierID, Name, Address
FROM Supplier
WHERE Name NOT LIKE '%Co%';
```

Output:

SupplierID	Name	Address
S2	La Maison du Marché	12 Rue Lafayette, Paris, France
S3	Nordic Exports AB	45 Kungsgatan, Stockholm, Sweden
S4	Café del Sol Imports	Calle 22, Madrid, Spain
S5	Silver Maple Supplies	221 King Street, Toronto, Canada
S6	Bavaria Goods GmbH	89 Hauptstraße, Munich, Germany

SQL BETWEEN Operator

The SQL BETWEEN operator is used to retrieve values that fall within a specified range. It works with numbers, dates, and text and makes range-based filtering simple and readable.

- Filters results between two values, including both boundaries.
- Works on numeric, date, and string columns.
- Useful for filtering data within defined ranges.

Example: First, we create a demo SQL database and table, on which we will use the BETWEEN Operator command.

EmpID	Name	Age
1	Liam	25
2	Sofia	30
3	Noah	35

Employees Table

Query:

```
SELECT * FROM Employees
WHERE Age BETWEEN 26 AND 35;
```

Output:

EmpID	Name	Age
2	Sofia	30
3	Noah	35

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Examples of BETWEEN Operator

To understand the SQL BETWEEN Operator we use the below below employees table with the various examples and their output.

EmployeeID	FirstName	LastName	Age	Salary	HireDate
1	John	Doe	28	50000	2021-01-15
2	Jane	Smith	34	60000	2020-03-22
3	Sam	Brown	45	75000	2018-07-10
4	Sue	Wilson	25	48000	2021-10-01
5	Tom	Harris	38	68000	2019-05-12

Employees Table

Example 1: NOT BETWEEN Text Values
Find employees whose last names are not alphabetically between 'B' and 'S'.

Query:

```
SELECT FirstName, LastName
FROM Employees
WHERE LastName NOT BETWEEN 'B' AND 'S';
```

Output:

FirstName	LastName
Jane	Smith
Sue	Wilson

- The query selects employees whose last names are lexicographically outside the range 'B' to 'S', based on full string comparison.
- Smith and Wilson are greater than 'S' in string order, so Jane Smith and Sue Wilson appear in the result.

Example 2: BETWEEN Dates
Find employees hired between January 1, 2020 and December 31, 2021.
Query:

```
SELECT FirstName, LastName, HireDate
FROM Employees
WHERE HireDate BETWEEN '2020-01-01' AND '2021-12-31';
```

Output:

FirstName	LastName	HireDate
John	Doe	2021-01-15
Jane	Smith	2020-03-22
Sue	Wilson	2021-10-01

- This query returns employees who were hired during the years 2020 and 2021.
- The BETWEEN operator is used to filter the HireDate field, returning records within the specified date range.

Example 3: NOT BETWEEN
Find employees whose age is not between 30 and 40.

Query:

```
SELECT FirstName, LastName, Age
FROM Employees
WHERE Age NOT BETWEEN 30 AND 40;
```

Output:

FirstName	LastName	Age
John	Doe	28
Sam	Brown	45
Sue	Wilson	25

- This query retrieves employees whose age does not fall between 30 and 40.
- The NOT BETWEEN operator is used here to exclude employees within that age range.
- John, Sam, and Sue meet this condition, as their ages are outside the 30-40 range.

Example 4: BETWEEN with IN

Find employees whose salaries are between 50,000 and 70,000 and whose first names are either 'John', 'Sue', or 'Tom'.

Query:

```
SELECT FirstName, LastName, Salary
FROM Employees
WHERE Salary BETWEEN 50000 AND 70000
  AND FirstName IN ('John', 'Sue', 'Tom');
```

Output:

FirstName	LastName	Salary
John	Doe	50000
Tom	Harris	68000

- Filters employees whose salary is between 50,000 and 70,000.
- Checks whether the first name is John, Sue, or Tom.
- Only John and Tom satisfy both conditions and appear in the result

SQL IN Operator

The IN operator in SQL is used to filter query results by checking whether a column's value matches any value in a specified list. It acts like a shorthand for writing multiple OR conditions, making queries cleaner and easier to read.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name IN (value1, value2, .....);
```

Example of SQL IN Operator

Let's understand IN Operator in SQL with examples. Here we will look at different examples. First, we create a demo SQL database and table, on which we will use the IN Operator command.

Fname	Lname	Ssn	Bdate	Address	Salary
Akira	Tanaka	1	2002-07-31	Tokyo, Japan	1111789
Luca	Rossi	2	1990-07-31	Rome, Italy	3333
Sofia	Müller	5	2002-04-04	Berlin, Germany	3433
Ethan	Johnson	6	2001-05-05	New York, USA	56564
Amélie	Dubois	7	2002-03-05	Paris, France	7565
Carlos	García	8	2002-08-04	Madrid, Spain	44657
Oliver	Brown	9	2003-11-10	London, UK	244322

Example 1: Basic Use of the IN Operator
SQL Query to get Fname and Lname of employees who have address in Tokyo, Japan, France and Paris.

Query:

```
SELECT Fname, Lname
FROM Employee
WHERE Address IN ('Tokyo, Japan', 'Paris, France');
```

Output:

Fname	Lname
Akira	Tanaka
Amélie	Dubois

- We fetched the Fname and Lname of employees whose Address is either Tokyo, Japan or Paris, France
- The IN operator was used in the WHERE clause to specify these multiple address values.

Example 2: SQL IN and NOT IN Operators
We can use the SQL IN with the NOT operator to exclude specified data from our result.
Query:

```
SELECT Fname
FROM Employee
WHERE Address NOT IN ('Tokyo, Japan', 'London, UK');
```

Output:

Fname
Luca
Sofia
Ethan
Amélie
Carlos

- We created a query to fetch the Fname (first name) of all employees.
- The condition is that the supplier's address should be neither 'Tokyo, Japan', 'London', nor 'UK'.
- To exclude these addresses, we used the NOT operator along with the IN clause.
- This ensures only employees with other addresses are displayed in the result.

Example 3: IN Operator with Subquery

We have used IN operator with explicit values for conditions. However, we can use the IN operator to select values for the condition from another query. For demonstrating this, we will use another table from the database, manager. The contents of the table are given below.

Ssn	Department
5	Sales
1	Technical

Now, we write a query to fetch details of all employees who are managers. This can be done by using nested SELECT queries with the IN operator.

Query:

```
SELECT * FROM employee
WHERE Ssn IN (SELECT Ssn FROM manager);
```

Output:

Fname	Lname	Department
Sofia	Müller	Sales
Akira	Tanaka	Technical

- We first selected the Ssn of all managers from the Manager table.
- This inner query (or nested query) returns a list of manager SSNs.
- We then used the IN operator in the main query to compare employee SSNs with those from the nested query.
- As a result, the main query fetches the details of all employees whose Ssn matches any Ssn returned by the manager query.
- This approach helps link data between two related tables using a subquery.

Note: *The IN operator returns TRUE if a value matches any item in the given list or subquery result.*

SQL Joins (Inner, Left, Right and Full Join)

SQL Joins are used to combine data from two or more tables based on a related column. They help in:

- Retrieving connected data stored across multiple tables.
- Matching records using common columns.
- Improving data analysis by combining related information.
- Creating meaningful result sets from separate tables.

Types of SQL Joins

SQL joins are categorized into different types based on how rows from two tables are matched and combined.

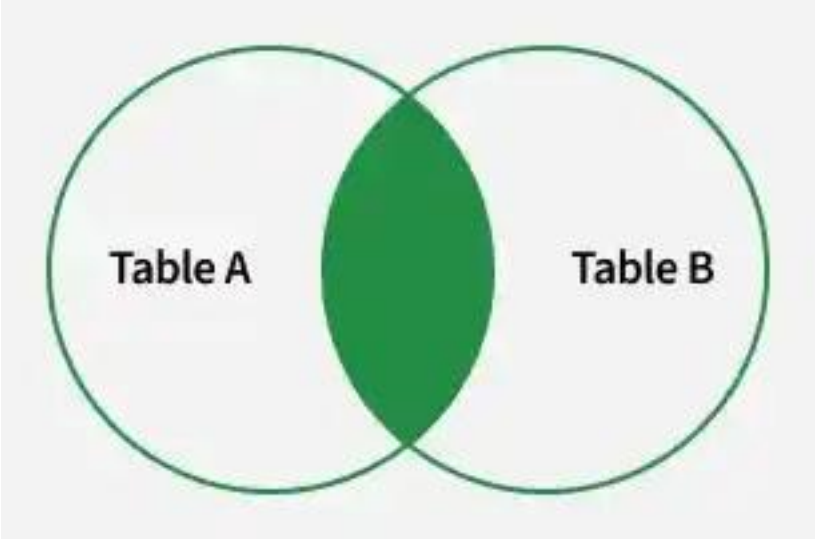
1. INNER JOIN

INNER JOIN is used to retrieve rows where matching values exist in both tables. It helps in:

- Combining records based on a related column.
- Returning only matching rows from both tables.
- Excluding non-matching data from the result set.
- Ensuring accurate data relationships between tables.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,.... FROM table1  INNER JOIN
 table2 ON  table1.matching_column = table2.matching_column;
```



Inner join

Note*: We can also write JOIN instead of INNER JOIN. JOIN is same as INNER JOIN.*

Example of INNER JOIN:

Consider the two tables, Student and StudentCourse, which share a common column ROLL_NO. Using SQL JOINS, we can combine data from these tables based on their relationship, allowing us to retrieve meaningful information like student details along with their enrolled courses.

Student Table:

ROLL_NO	Name	Address	Age
1	Liam	London	18
2	Sofia	Berlin	19
3	Mateo	Madrid	20
4	Emma	Paris	21
5	Noah	Rome	24

StudentCourse Table:

COURSE_ID	ROLL_NO
1	1
2	2
3	3
6	7

The following example demonstrates the working of the INNER JOIN clause. This query displays the names and ages of students enrolled in different courses.

Query:

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE
FROM Student
INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

Output:

Course_ID	Name	Age
1	Liam	18
2	Sofia	19
3	Mateo	20

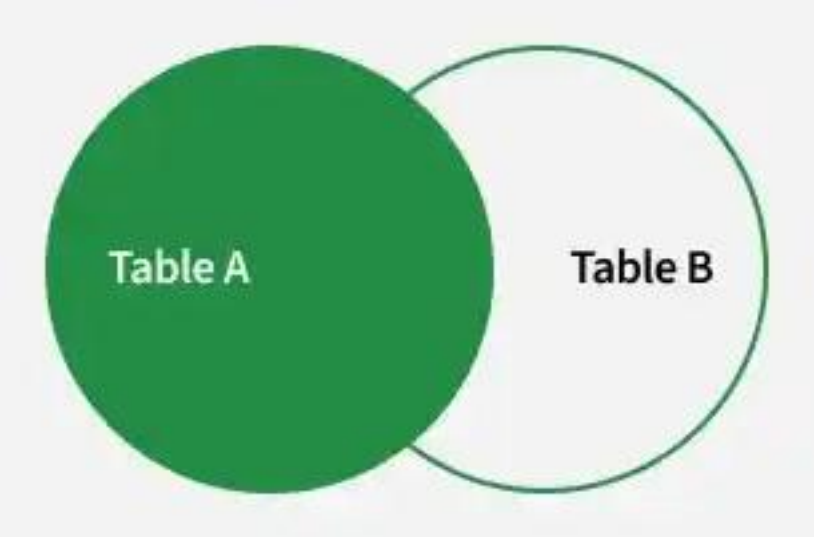
2. LEFT JOIN

LEFT JOIN is used to retrieve all rows from the left table and matching rows from the right table. It helps in:

- Returning all records from the left table.
- Showing matching data from the right table.
- Displaying NULL values where no match exists in the right table.
- Performing outer joins, also known as LEFT OUTER JOIN.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```



Left Join

***Note**: We can also use LEFT OUTER JOIN instead of LEFT JOIN, both are the same.*
Example: In this example, the LEFT JOIN retrieves all rows from the Student table and the matching rows from the StudentCourse table based on the ROLL_NO column.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output:

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
Olivia Brown	
Ava Davis	

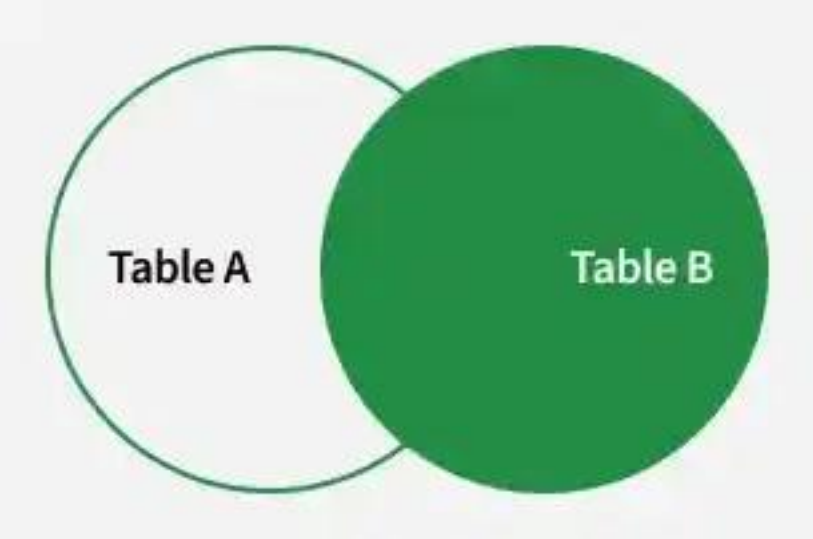
3. RIGHT JOIN

RIGHT JOIN is used to retrieve all rows from the right table and the matching rows from the left table. It helps in:

- Returning all records from the right-side table.
- Showing matching data from the left-side table.
- Displaying NULL values where no match exists in the left table.
- Performing outer joins, also known as RIGHT OUTER JOIN.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```



Right Join

***Note**: We can also use RIGHT OUTER JOIN instead of RIGHT JOIN, both are the same*

Example: In this example, the RIGHT JOIN retrieves all rows from the StudentCourse table and the matching rows from the Student table based on the ROLL_NO column.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
RIGHT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output:

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
	6

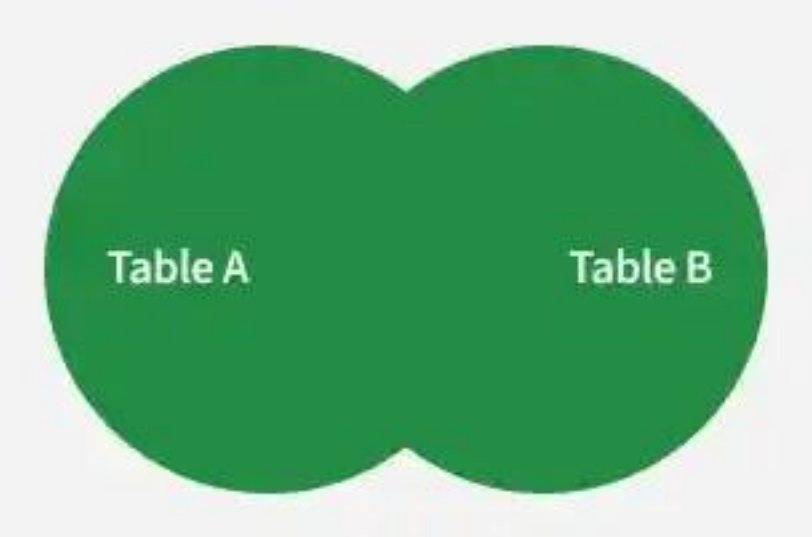
4. FULL JOIN

FULL JOIN is used to combine the results of both LEFT JOIN and RIGHT JOIN. It helps in:

- Returning all rows from both tables.
- Showing matching records from each table.
- Displaying NULL values where no match exists in either table.
- Providing complete data from both sides of the join.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```



Full Join

Example: This example uses a FULL JOIN to return all rows from both tables. Matching records appear together, while non-matching records still show up with NULL values for the missing fields.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
FULL JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output :

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
Olivia Brown	
Ava Davis	
	6

Note: *MySQL does not support FULL OUTER JOIN directly. It cab be simulated using UNION of LEFT JOIN and RIGHT JOIN.*

5. Natural Join

A Natural Join is a type of INNER JOIN that automatically joins two tables based on columns with the same name and data type. It returns only the rows where the values in the common columns match.

- It joins tables using common columns with the same name.
- It returns only rows where values in those columns match.
- The common column appears only once in the result.

Example:

Employee Table

Emp_id	Emp_name	Dept_id
1	Ram	10
2	Jon	30
3	Bob	50

Department Table

Dept_id	Dept_name
10	IT
30	HR
40	TIS

Example: Find all Employees and their respective departments.

```
SELECT
   Emp_name,
   Dept_name
FROM Employee
NATURAL JOIN Department;
```

Output:

Emp_name	Dept_name
Ram	IT
Jon	HR

Views:

An SQL view is a virtual table based on the result-set of an SQL statement. An SQL view contains rows and columns, just like a real table. The fields in the view are fields from one or more real tables in the database.

You can add SQL statements and functions to a view and present the data as if it were coming from one single table.

A view is created with the CREATE VIEW statement.

CREATE VIEW Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Note: A view always shows real-time data! The database engine only stores the view's definition (the SELECT statement), not a copy of the data.

SQL View is a virtual table created from the result of a SELECT query. It does not store data physically but displays data stored in underlying tables. Views help simplify complex queries, enhance security, and present data in a cleaner, customized format.

Example: First, we will create a demo SQL database and table, on which we will use the TRUNCATE TABLE command.

S_ID	NAME	ADDRESS
1	Liam	New York
2	Sophia	London
3	Noah	Toronto

```
CREATE VIEW StudentView AS
SELECT NAME, ADDRESS
FROM StudentDetails;
SELECT * FROM StudentView;
```

Output:

NAME	ADDRESS
Liam	New York
Sophia	London
Noah	Toronto

CREATE VIEWS in SQL

We can create a view using CREATE VIEW statement. A View can be created from a single table or multiple tables.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_name
WHERE condition;
```

- **view_name**: Name for the View
- **table_name**: Name of the table
- **condition**: Condition to select rows

Example 1: Creating a Simple View from a Single Table

First, we will create a demo SQL database and table, on which we will use the View command.

S_ID	NAME	ADDRESS
1	Liam	London
2	Sofia	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome

In this example, we create a View named DetailsView from the table StudentDetails.

Query:

```
CREATE VIEW DetailsView AS
SELECT NAME, ADDRESS
FROM StudentDetails
WHERE S_ID < 5;
```

Use the below query to retrieve the data from this view
SELECT * FROM DetailsView;

Output:

NAME	ADDRESS
Liam	London
Sofia	Berlin
Mateo	Madrid
Emma	Paris

Example 2: Creating a View From Multiple Tables

In this example, we create a View MarksView that combines data from bothtables StudentDetails and StudentMarks. To create a View from multiple tables we can simply include multiple tables in the SELECT statement.

NAME	MARKS
Liam	85
Sofia	92
Mateo	78
Emma	88

Query:

```
CREATE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

To display data of View MarksView:

```
SELECT * FROM MarksView;
```

Output:

NAME	ADDRESS	MARKS
Liam	London	85
Sofia	Berlin	92
Mateo	Madrid	78
Emma	Paris	88

Managing Views

Here are some common operations used to manage views in SQL:

1. Listing all Views in a Database

We can list all the views in a database using the SHOW FULL TABLES statement or by querying the information_schema tables.

```
USE "database_name";
SHOW FULL TABLES WHERE table_type LIKE "%VIEW";
```

Using information_schema

```
SELECT table_name
FROM information_schema.views
WHERE table_schema = 'database_name';
```

OR

```
SELECT table_schema, table_name, view_definition
FROM information_schema.views
WHERE table_schema = 'database_name';
```

2. Deleting a View

SQL allows us to delete an existing View. We can delete or drop View using the DROP statement.

Syntax:

```
DROP VIEW view_name;
```

Example: In this example, we are deleting the View MarksView.

```
DROP VIEW MarksView;
```

3. Updating a View Definition

If we want to update the existing data within the view, use the **UPDATE** statement.

```
UPDATE view_name
SET column1 = value1, column2 = value2...., columnN = valueN
WHERE [condition];
```

If you want to update the view definition without affecting the data, use the CREATE OR REPLACE VIEW statement. For example, let's add the Age column to the MarksView:

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Note: Not all views can be updated using the UPDATE statement.

Rules to Update Views in SQL:

Certain conditions need to be satisfied to update a view. If any of these conditions are **not** met, the view can not be updated.

1. The SELECT statement which is used to create the view should not include GROUP BY clause or ORDER BY clause.
2. The SELECT statement should not have the DISTINCT keyword.
3. The View should have all NOT NULL values.
4. The view should not be created using nested queries or complex queries.
5. The view should be created from a single table. If the view is created using multiple tables then we will not be allowed to update the view.

Advanced Techniques with Views

Here are some advanced ways to work with SQL views:

1. Updating Data Through Views

We can use the CREATE OR REPLACE VIEW statement to add or replace fields from a view If we want to update the view MarksView and add the field AGE to this View from StudentMarks Table, we can do this by:

Example:

```
CREATE OR REPLACE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS,
StudentMarks.AGE
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

If we fetch all the data from MarksView now as:

```
SELECT * FROM MarksView;
```

Output:

NAME	S_ID	ADDRESS	MARKS
Liam	1	London	85
Sofia	2	Berlin	92
Mateo	3	Madrid	78
Emma	4	Paris	88

2. Inserting Data into Views

We can insert a row in a View in the same way as we do in a table. We can use the INSERT INTO statement of SQL to insert a row in a View. In the below example, we will insert a new row in the View StudentDetailswhich we have created above in the example of "creating views from a single table".

Example:

```
INSERT INTO StudentDetails(NAME, ADDRESS)
VALUES("John","German");
```

If we fetch all the data from DetailsView now as,

```
SELECT * FROM StudentDetails;
```

Output:

NAME	ADDRESS
Liam	London
Sofia	Berlin
Mateo	Madrid
Emma	Paris
Noah	Rome
John	German

3. Deleting a row from a View

Deleting rows from a view works the same as deleting from a table. Using the DELETE command removes the row from the base table, and the change automatically appears in the view. In this example, we delete the last row from StudentDetails added earlier.

Query:

```
DELETE FROM StudentDetails
WHERE NAME="John";
```

If we fetch all the data from DetailsView now as,

```
SELECT * FROM StudentDetails;
```

Output:

NAME	ADDRESS
Liam	London
Sofia	Berlin
Mateo	Madrid
Emma	Paris
Noah	Rome

4. WITH CHECK OPTION Clause

The WITH CHECK OPTION clause ensures that any INSERT or UPDATE on an updatable view must satisfy the view's WHERE condition. If the condition is violated, SQL returns an error. In this example, a view is created with WITH CHECK OPTION to restrict changes that fall outside the defined criteria.

Query:

```
CREATE VIEW SampleView AS
SELECT S_ID, NAME
FROM  StudentDetails
WHERE NAME IS NOT NULL
WITH CHECK OPTION;
UPDATE SampleView SET NAME = 'Mark' WHERE S_ID = 2;
```

Output:

S_ID	NAME	ADDRESS
1	Liam	London
2	Mark	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome